

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Data Interpretation

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 1 – Data Interpretation
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	SYED MUHAFIZ A	Reg. No.:	192472282
Section / Batch:		Date:	

Code of Conduct

I SYED MUHAFIZ A (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Instructions

- Read each data set, table, semantic representation, or scenario carefully before answering.
- Analyze the given information and provide logical, evidence-based interpretations.
- Show all intermediate reasoning, semantic analysis steps, predicate representations, or calculations wherever applicable.
- Answers should be concise, structured, and technically accurate.
- Support your conclusions using the data provided in the question.
- Draw diagrams, semantic networks, parse trees, or logical representations wherever relevant.
- Avoid copying definitions alone; focus on analyzing the given scenario and interpreting the data.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic representations, identify action–entity relationships, detect interpretation errors, and evaluate meaning representation in chatbot systems.	Q1
First-Order Predicate Calculus & Representing Linguistically Relevant Concepts	Construct predicate representations, apply logical inference rules, derive conclusions, and evaluate reasoning in smart manufacturing environments.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze contextual clues, determine correct word senses, evaluate ambiguity resolution, and improve semantic understanding in search systems.	Q3
Syntax-Driven Semantic Analysis	Analyze syntactic structures, assign semantic roles, evaluate parsing accuracy, and improve semantic interpretation in healthcare NLP applications.	Q4

Annexure A – Data Interpretation

1. Semantic Representation in Customer Support Chatbots.

A telecom company collected customer chatbot interactions. The semantic analyzer generated the following meaning representations.

Customer Query	Semantic Representation
"Activate international roaming for my number."	ACTIVATE(Roaming, Customer)
"Deactivate caller tune service."	DEACTIVATE(CallerTune, Customer)
"Check my data balance."	QUERY(DataBalance, Customer)
"Enable 5G service."	ACTIVATE(5GService, Customer)

Additional chatbot logs show:

Query ID	Actual User Intent	Predicted Intent
Q1	Activate Roaming	Activate Roaming
Q2	Deactivate Caller Tune	Activate Caller Tune
Q3	Check Data Balance	Query Data Balance
Q4	Enable 5G Service	Activate 5G Service

Tasks:

1. Analyze the semantic representations and identify the action-object relationships.
 2. Determine which query contains semantic interpretation errors.
 3. Evaluate how meaning representation affects chatbot decision accuracy.
 4. Recommend improvements to enhance semantic understanding in telecom customer support systems.
2. First-Order Predicate Calculus for Smart Manufacturing.
- An Industry 4.0 manufacturing plant monitors machines using semantic rules.

Production Data:

Machine	Status
M1	Active
M2	Active
M3	Maintenance
M4	Active

Predicate Rules:

- $\text{Active}(x) \rightarrow \text{Producing}(x)$
- $\text{Produces}(x,y) \wedge \text{Active}(x) \rightarrow \text{Available}(y)$
- $\text{Maintenance}(x) \rightarrow \neg \text{Producing}(x)$

Tasks:

1. Represent the production data using First-Order Predicate Calculus.
 2. Analyze which products are currently available.
 3. Infer whether Gear production is affected by maintenance activities.
 4. Evaluate the effectiveness of predicate logic in industrial decision-support systems.
3. Word Sense Disambiguation in E-Commerce Search Engines.
- An e-commerce company observed the following search logs.

User Query	Possible Sense 1	Possible Sense 2
"Apple accessories"	Fruit	Technology Brand

"Mouse wireless"	Animal	Computer Device
"Java tutorial"	Island	Programming Language
"Python course"	Snake	Programming Language

Search Context Data:

Query	Clicked Result
Apple accessories	iPhone Charger
Mouse wireless	Bluetooth Mouse
Java tutorial	Coding Lessons
Python course	Software Development Training

Tasks:

1. Analyze the contextual information and determine the correct word sense for each query.
2. Identify the semantic cues used for disambiguation.
3. Evaluate the impact of incorrect sense selection on search engine performance.
4. Suggest an industrial-scale WSD strategy for improving recommendation accuracy.

4. Syntax-Driven Semantic Analysis in Healthcare Information Systems.

A healthcare NLP system processes medical reports.

Parsed Sentences:

Sentence	Parse Structure
Doctor prescribed medicine to patient.	Subject-Verb-Object
Patient reported severe headache.	Subject-Verb-Object
Nurse monitored patient continuously.	Subject-Verb-Object
Medicine reduced blood pressure.	Subject-Verb-Object

Semantic Roles Generated:

Entity	Role
Doctor	Agent
Medicine	Instrument
Patient	Recipient
Headache	Symptom

Tasks:

1. Analyze how syntactic structures contribute to semantic role assignment.
2. Determine whether the assigned semantic roles are appropriate.
3. Evaluate potential errors that could arise from incorrect parsing.
4. Recommend methods to improve syntax-driven semantic analysis in healthcare applications.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Understanding of Data	Analysis	Application of Concepts	Conclusion	Clarity	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Semantic Representation in Customer Support Chatbots

Telecom chatbots rely on a semantic analyzer that converts free-text customer queries into a formal meaning representation of the form ACTION(OBJECT, AGENT). This representation strips away surface-level syntactic variation and exposes the underlying intent that downstream business logic (billing systems, provisioning APIs, CRM workflows) can act upon.

1.0 Given Data

Customer Query	Semantic Representation
"Activate international roaming for my number."	ACTIVATE(Roaming, Customer)
"Deactivate caller tune service."	DEACTIVATE(CallerTune, Customer)
"Check my data balance."	QUERY(DataBalance, Customer)
"Enable 5G service."	ACTIVATE(5GService, Customer)

Table 1.1 – Semantic representations generated by the chatbot analyzer

Query ID	Actual User Intent	Predicted Intent
Q1	Activate Roaming	Activate Roaming
Q2	Deactivate Caller Tune	Activate Caller Tune
Q3	Check Data Balance	Query Data Balance
Q4	Enable 5G Service	Activate 5G Service

Table 1.2 – Chatbot intent prediction logs

1.1 Task 1 — Action–Object Relationships

Every semantic representation follows a consistent predicate-argument structure: a two-place (or, when the implicit agent is included, three-place) predicate whose first argument is the OBJECT being acted upon and whose second argument is the AGENT initiating the action. The action itself is encoded as the predicate name, drawn from a closed set of chatbot-recognised intents.

Predicate (Action)	Object Argument	Agent Argument	Interpreted Meaning
ACTIVATE	Roaming	Customer	Customer requests the Roaming service be switched ON
DEACTIVATE	CallerTune	Customer	Customer requests the CallerTune service be switched OFF
QUERY	DataBalance	Customer	Customer requests a read-only status/value lookup
ACTIVATE	5GService	Customer	Customer requests the 5G service be switched ON

Table 1.3 – Decomposition of predicate–argument (action–object) structure

Two broad predicate classes emerge from this decomposition: (a) State-Change predicates — ACTIVATE and DEACTIVATE — which mutate a service flag in the provisioning system from OFF↔ON, and (b) Read-Only predicates — QUERY — which retrieve a value without altering system state. This

binary classification is itself a critical semantic feature, because a state-change predicate that is mis-recognised can trigger an irreversible billing or provisioning action, whereas a mis-recognised query predicate merely returns the wrong information.

1.2 Task 2 — Identification of Semantic Interpretation Errors

Comparing the Actual User Intent against the Predicted Intent column reveals that the four logged queries do not all carry the same severity of error:

1. Q1 (Activate Roaming → Activate Roaming): No error. Predicate, object and polarity all match; this is a correctly resolved query.
2. Q2 (Deactivate Caller Tune → Activate Caller Tune): Critical polarity error. The analyzer correctly identified the object (CallerTune) but inverted the action — DEACTIVATE was misclassified as ACTIVATE. This is the most serious class of error because it produces the semantically opposite outcome of what the customer requested (the service would be turned ON when the customer wanted it turned OFF), directly causing incorrect billing and customer dissatisfaction.
3. Q3 (Check Data Balance → Query Data Balance): Surface-level lexical mismatch, not a true semantic error. "Check" and "Query" are synonymous read-only intents that map to the same underlying predicate QUERY(DataBalance, Customer); provided the intent taxonomy normalises both labels to one canonical predicate, the outcome delivered to the customer is unaffected.
4. Q4 (Enable 5G Service → Activate 5G Service): Similarly a lexical/label variation rather than a semantic error. "Enable" and "Activate" are synonyms for the same state-change predicate ACTIVATE(5GService, Customer); the resulting system action is correct even though the label string differs.

Conclusion: only Q2 contains a genuine semantic interpretation error (polarity/action inversion). Q3 and Q4 illustrate the more common and lower-risk problem of intent-label proliferation — synonymous surface forms not yet mapped onto a single canonical predicate — which affects reporting/analytics consistency but not the correctness of the action taken, so long as canonicalisation is enforced before execution.

1.3 Task 3 — Impact of Meaning Representation on Decision Accuracy

The chatbot's downstream decision layer acts directly on the predicate-argument tuple, not on the raw text, so the fidelity of the meaning representation is the single largest determinant of execution accuracy:

- Predicate correctness gates the action taken: an inverted predicate (as in Q2) causes the system to execute the logical opposite of the customer's request — a false-positive activation coupled with a false-negative deactivation.
- Argument correctness gates the target of the action: even a correctly classified predicate is useless if the object argument (e.g., CallerTune vs. Roaming) is mis-extracted, since the wrong service would be toggled.
- Canonicalisation of synonymous predicates (Check/Query, Enable/Activate) affects consistency of downstream analytics and confidence scoring rather than correctness of the action, but inconsistent labelling can still degrade model retraining if treated as separate classes during supervised learning.
- Because ACTIVATE/DEACTIVATE are state-changing and largely irreversible without a follow-up correction, the cost of a polarity error (Q2) is asymmetrically higher than the cost of a QUERY misclassification, which only risks displaying a wrong read-only value.

Overall, chatbot decision accuracy in this domain is bounded not by surface NLU (whether the words were understood) but by whether the extracted meaning representation preserves polarity, object identity and agent scope with perfect fidelity — a single flipped predicate is sufficient to produce a customer-facing service failure even when 100% of the vocabulary was recognised correctly.

1.4 Task 4 — Recommendations to Improve Semantic Understanding

5. Canonical Intent Ontology: Maintain a controlled vocabulary that maps all lexical variants (Check/Query, Enable/Activate, Turn on/Switch on) to a single canonical predicate before the decision layer consumes the representation, eliminating false "errors" caused purely by labelling drift.
6. Polarity-Aware Training Data Augmentation: Deliberately oversample negation and deactivation phrasings ("stop", "remove", "cancel", "turn off") during model training/fine-tuning, since Q2 shows polarity inversion is the highest-risk failure mode.
7. Confidence-Gated Execution with Confirmation: For all ACTIVATE/DEACTIVATE (state-changing) predicates below a confidence threshold, require an explicit "Did you mean to deactivate X?" confirmation step before committing the action, while QUERY predicates can execute without confirmation given their low risk.
8. Contextual Embedding Models: Replace purely lexical intent classifiers with transformer-based contextual embeddings (e.g., BERT-family encoders) fine-tuned on telecom-domain dialogues, which are far more robust to negation and polarity than bag-of-words or keyword-matching approaches.
9. Continuous Human-in-the-Loop Auditing: Route a sample of low-confidence and all state-changing predictions to human reviewers, feeding corrected labels back into periodic retraining so that systemic errors like the Q2 polarity flip are caught and corrected quickly.
10. Post-Hoc Semantic Consistency Checks: Before execution, validate the predicted predicate against the customer's account state (e.g., block a DEACTIVATE prediction for a service that is already inactive, or an ACTIVATE for one already active) as a sanity filter that can catch some polarity errors automatically.

2. First-Order Predicate Calculus for Smart Manufacturing

Industry 4.0 plants increasingly encode machine status and production dependencies as First-Order Predicate Calculus (FOPC) facts and rules, enabling automated reasoning engines to infer product availability and diagnose the downstream effect of a machine going into maintenance, without a human having to trace the production graph manually.

2.0 Given Data

Machine	Status
M1	Active
M2	Active
M3	Maintenance
M4	Active

Table 2.1 – Machine production status

Predicate Rules:

- $\text{Active}(x) \rightarrow \text{Producing}(x)$
- $\text{Produces}(x, y) \wedge \text{Active}(x) \rightarrow \text{Available}(y)$
- $\text{Maintenance}(x) \rightarrow \neg \text{Producing}(x)$

Note on assumption: the source data does not specify which product each machine manufactures. To carry out Tasks 2 and 3 concretely, the following reasonable and explicitly-stated mapping is assumed, reflecting a common plant configuration in which more than one machine can produce the same part for redundancy:

Machine	Product (assumed)
M1	Gear
M2	Bolt
M3	Gear
M4	Screw

Table 2.2 – Assumed $\text{Produces}(\text{machine}, \text{product})$ mapping used for inference

2.1 Task 1 — FOPC Representation of Production Data

The plant state is first represented as a set of ground atomic facts:

- $\text{Active}(\text{M1}), \text{Active}(\text{M2}), \text{Active}(\text{M4})$
- $\text{Maintenance}(\text{M3})$
- $\text{Produces}(\text{M1}, \text{Gear}), \text{Produces}(\text{M2}, \text{Bolt}), \text{Produces}(\text{M3}, \text{Gear}), \text{Produces}(\text{M4}, \text{Screw})$

Combined with the three universally quantified rules, the full knowledge base is:

- $\forall x [\text{Active}(x) \rightarrow \text{Producing}(x)]$
- $\forall x \forall y [\text{Produces}(x, y) \wedge \text{Active}(x) \rightarrow \text{Available}(y)]$
- $\forall x [\text{Maintenance}(x) \rightarrow \neg \text{Producing}(x)]$

This representation cleanly separates transient state (Active/Maintenance, which changes shift-to-shift) from structural facts (Produces, which changes only when the plant is re-tooled), which is precisely what makes FOPC attractive for industrial reasoning: the rules never need to be rewritten when a machine's status changes — only the ground facts are updated and the inference engine re-fires automatically.

2.2 Task 2 — Products Currently Available

Applying Rule 1 ($\text{Active}(x) \rightarrow \text{Producing}(x)$) to the ground facts:

- $\text{Active}(M1) \rightarrow \text{Producing}(M1)$
- $\text{Active}(M2) \rightarrow \text{Producing}(M2)$
- $\text{Active}(M4) \rightarrow \text{Producing}(M4)$
- M3 does not satisfy $\text{Active}(x)$; instead $\text{Maintenance}(M3)$ holds, so by Rule 3, $\neg\text{Producing}(M3)$ is derived directly.

Applying Rule 2 ($\text{Produces}(x,y) \wedge \text{Active}(x) \rightarrow \text{Available}(y)$) using only machines for which $\text{Active}(x)$ holds:

- $\text{Produces}(M1, \text{Gear}) \wedge \text{Active}(M1) \rightarrow \text{Available}(\text{Gear})$
- $\text{Produces}(M2, \text{Bolt}) \wedge \text{Active}(M2) \rightarrow \text{Available}(\text{Bolt})$
- $\text{Produces}(M4, \text{Screw}) \wedge \text{Active}(M4) \rightarrow \text{Available}(\text{Screw})$
- $\text{Produces}(M3, \text{Gear}) \wedge \text{Active}(M3)$ cannot be derived because $\text{Active}(M3)$ is false (M3 is in Maintenance) — this instance of the rule simply does not fire for M3.

Therefore the set of currently Available products is { Gear, Bolt, Screw }. Gear remains available only because it is redundantly produced by M1, which is Active; had Gear been produced solely by M3, $\text{Available}(\text{Gear})$ would not be derivable at all.

2.3 Task 3 — Is Gear Production Affected by Maintenance?

This question must be answered at two levels — machine level and product level — because FOPC reasoning distinguishes clearly between the two:

- At the machine level: Yes. M3 is explicitly in Maintenance, so by Rule 3, $\neg\text{Producing}(M3)$ is proven. M3's contribution to Gear output is fully suspended for as long as $\text{Maintenance}(M3)$ holds.
- At the product level: No (under the assumed mapping). Because M1 is also configured to produce Gear and M1 is Active, $\text{Available}(\text{Gear})$ is still derivable via M1 alone. The plant-wide Gear supply is therefore not interrupted — the redundancy absorbs the loss of M3's output.
- Risk flag: this conclusion is fragile. If M1 were to also enter Maintenance (or if Gear had only ever been produced by M3), $\text{Available}(\text{Gear})$ would become non-derivable, and the reasoning engine would correctly flag a stock-out risk. A capacity-aware extension of the rule base (tracking quantity, not just a boolean $\text{Available}/\neg\text{Available}$) would be needed to detect this before it becomes critical, since two machines being nominally "Active" does not guarantee sufficient combined throughput.

This illustrates a general strength of predicate-calculus modelling of manufacturing dependencies: it makes single points of failure and redundancy paths explicit and machine-verifiable, rather than left implicit in an engineer's mental model of the shop floor.

2.4 Task 4 — Effectiveness of Predicate Logic in Industrial Decision Support

Strengths

- Formal soundness: conclusions such as Available(Gear) are logically guaranteed to follow from the stated facts, giving auditable, explainable decisions — a significant advantage over opaque black-box models in safety- and compliance-sensitive manufacturing environments.
- Modularity: rules (Active→Producing, Produces∧Active→Available, Maintenance→¬Producing) are stated once and apply plant-wide; adding new machines only requires new ground facts, not new rules.
- Fast, incremental re-inference: when a machine's status changes (e.g., M3 exits Maintenance), only the affected facts are updated and forward-chaining re-derives Available(Gear)'s status in milliseconds — well suited to real-time SCADA/MES integration.
- Traceable root-cause analysis: because every derived fact carries a proof (which rule and which premises produced it), engineers can ask "why is Gear unavailable?" and get a precise causal chain rather than a statistical correlation.

Limitations

- Binary/qualitative reasoning only: standard FOPC as specified here has no notion of quantity, throughput rate, or time — it can say Gear is Available but not how many units per hour, which is usually what a plant manager actually needs.
- Closed-world / completeness assumption: the engine can only reason over facts explicitly asserted (e.g., the Produces mapping); an unstated or incorrectly captured Produces fact silently breaks the whole inference chain, as shown by the necessary assumption made in Task 1 above.
- Scalability of manual rule authoring: as the number of machines, products, and cross-dependencies (shared tooling, shared operators, supply-chain inputs) grows, hand-written FOPC rule sets become harder to maintain and may need to be paired with rule-mining or probabilistic extensions (e.g., Markov Logic Networks) to stay tractable.

On balance, predicate logic is highly effective as the deterministic reasoning backbone of an industrial decision-support system — particularly for availability, dependency and root-cause questions — but is best deployed alongside quantitative/statistical layers (forecasting throughput, predictive maintenance scoring) that handle the continuous, uncertain aspects that pure FOPC cannot express natively.

3. Word Sense Disambiguation in E-Commerce Search Engines

Search queries in e-commerce are frequently short, context-poor, and lexically ambiguous. Word Sense Disambiguation (WSD) uses surrounding query terms and, critically, behavioural signals such as click-through data to resolve which sense of an ambiguous term the user intended, directly shaping which product catalogue is surfaced.

3.0 Given Data

User Query	Possible Sense 1	Possible Sense 2
"Apple accessories"	Fruit	Technology Brand
"Mouse wireless"	Animal	Computer Device
"Java tutorial"	Island	Programming Language
"Python course"	Snake	Programming Language

Table 3.1 – Candidate senses per query

Query	Clicked Result
Apple accessories	iPhone Charger
Mouse wireless	Bluetooth Mouse
Java tutorial	Coding Lessons
Python course	Software Development Training

Table 3.2 – Search context / click-through data

3.1 Task 1 — Correct Word Sense per Query

Query	Resolved Sense	Evidence
Apple accessories	Technology Brand	Click on "iPhone Charger" is an Apple Inc. product
Mouse wireless	Computer Device	Click on "Bluetooth Mouse" is a peripheral, and "wireless" strongly co-occurs with electronics
Java tutorial	Programming Language	Click on "Coding Lessons" confirms the software-learning sense, not the island
Python course	Programming Language	Click on "Software Development Training" confirms the language sense, not the animal

Table 3.3 – Disambiguation outcome

In every one of the four cases, the second listed sense (Technology Brand, Computer Device, Programming Language, Programming Language) is the correct resolution, and in every case it is the click-through data — not the query text alone — that provides the disambiguating evidence, since the bare query strings are genuinely ambiguous without it.

3.2 Task 2 — Semantic Cues Used for Disambiguation

11. Co-occurring modifier words within the query itself: "accessories" and "wireless" are technology-domain collocates (fruits do not have "accessories"; animals are not described as "wireless"), so

even before any click data arrives, local lexical co-occurrence already biases toward the technology sense.

12. Domain/platform prior: the query is issued on an e-commerce electronics-heavy platform, so the prior probability of the technology sense is inflated by the surrounding catalogue context, independent of the specific words used.
13. Click-through (behavioural) signal: the strongest and most direct cue — what the user actually clicked after searching is a post-hoc confirmation of intended sense, and aggregated over many users becomes a highly reliable training signal for a WSD classifier.
14. Suffix/genre words: "tutorial" and "course" are strong indicators of an educational-content or skill-learning intent, which co-occurs overwhelmingly with the Programming-Language senses of Java/Python rather than the Island/Snake senses in an e-commerce/learning context.
15. Session and user-history context (implicit): repeat searches or category filters previously used by the same user (e.g., prior visits to an "Electronics" or "Online Courses" category) further narrow the sense space, although this is not explicitly present in the given log.

3.3 Task 3 — Impact of Incorrect Sense Selection on Search Performance

- Irrelevant result sets: selecting "Fruit" for "Apple accessories" would surface grocery items to a customer wanting phone chargers, producing a zero-relevance result page.
- Depressed click-through rate (CTR) and conversion: irrelevant top results are rarely clicked or purchased, directly harming the platform's core revenue metric.
- Erosion of user trust and increased query abandonment: repeated poor disambiguation trains users to distrust search and instead use manual category filters or leave the platform, a leading indicator of churn.
- Corrupted personalisation/ranking-model training data: if incorrect-sense clicks (or absence of clicks) are fed back into the ranking model without correction, the model can reinforce the wrong sense over time — a feedback loop that compounds the original error.
- Wasted advertising/inventory spend: sponsored placements and inventory promotion tied to the wrong sense (e.g., promoting fruit baskets against an electronics-intent query) wastes advertiser budget and misallocates merchandising shelf-space.

3.4 Task 4 — Industrial-Scale WSD Strategy

16. Contextual embedding models: deploy transformer-based encoders (e.g., BERT/Sentence-BERT variants) fine-tuned on the platform's own query-click corpus, since these capture polysemy far better than static word embeddings or keyword rules.
17. Click-log-supervised training: use aggregated, de-identified click-through and add-to-cart signals as weak-supervision labels, allowing sense disambiguation models to be retrained continuously as language and catalogue trends shift, without manual re-labelling at scale.
18. Knowledge-graph grounding: link ambiguous query terms to a product/category knowledge graph (e.g., "Apple" → Brand node vs. Produce node) so disambiguation can leverage structured catalogue relationships, not just text co-occurrence.
19. Category and session-context features: incorporate the active category filter, browsing history, and device/app context as additional disambiguation features fed into the ranking model, mirroring how a human agent would use conversational context.

20. A/B-tested confidence thresholds with fallback UI: when sense confidence is below threshold, surface a lightweight disambiguation prompt ("Did you mean: Apple (brand) or fresh apples?") rather than silently guessing, converting an ambiguous query into an explicit signal.
21. Continuous feedback loop and drift monitoring: track sense-resolution accuracy and CTR by query cluster over time, alerting when a term's dominant sense shifts (e.g., a new product launch changes what "Java" tends to mean on the platform) so the model can be retrained proactively rather than reactively.

4. Syntax-Driven Semantic Analysis in Healthcare Information Systems

Clinical NLP pipelines parse medical report sentences into syntactic structures (typically Subject–Verb–Object) and then map syntactic positions onto semantic roles (Agent, Instrument, Recipient/Patient, Symptom, etc.), because downstream clinical decision support (drug interaction checks, adverse-event detection, cohort identification) depends on knowing who did what to whom, not merely which words appeared in the sentence.

4.0 Given Data

Sentence	Parse Structure
Doctor prescribed medicine to patient.	Subject–Verb–Object
Patient reported severe headache.	Subject–Verb–Object
Nurse monitored patient continuously.	Subject–Verb–Object
Medicine reduced blood pressure.	Subject–Verb–Object

Table 4.1 – Parsed sentences

Entity	Role
Doctor	Agent
Medicine	Instrument
Patient	Recipient
Headache	Symptom

Table 4.2 – Semantic roles generated by the system

4.1 Task 1 — How Syntax Contributes to Semantic Role Assignment

The Subject–Verb–Object (SVO) template gives a strong, though not infallible, first-pass mapping onto semantic roles: the grammatical Subject typically maps to the Agent (the entity performing/initiating the action), the main Verb encodes the predicate/action type, and the grammatical Object typically maps to a Theme, Recipient or Patient (the entity affected by the action), with prepositional phrases (e.g., "to patient") often marking a Recipient/Beneficiary distinct from a direct-object Theme.

Sentence	Subject → Role	Verb → Predicate	Object/PP → Role
Doctor prescribed medicine to patient.	Doctor → Agent	prescribed	medicine → Theme; "to patient" → Recipient
Patient reported severe headache.	Patient → Agent (Experiencer)	reported	severe headache → Theme/Symptom
Nurse monitored patient continuously.	Nurse → Agent	monitored	patient → Theme/Patient(clinical)
Medicine reduced blood pressure.	Medicine → Agent/Instrument(cause)	reduced	blood pressure → Theme (affected attribute)

Table 4.3 – Syntax-to-role mapping across the four sentences

This mapping works reliably when the verb is a canonical transitive action verb with a clear human Agent (as in "Doctor prescribed medicine"), but becomes less reliable as the Subject moves away from being a volitional human actor — as seen directly in Task 2 below.

4.2 Task 2 — Are the Assigned Semantic Roles Appropriate?

Evaluating each of the four given role assignments against standard semantic role labelling (SRL) conventions (e.g., PropBank-style Agent/Theme/Recipient/Instrument):

- Doctor → Agent: Appropriate. The Doctor is the volitional, human initiator of the "prescribed" action — a textbook Agent role.
- Patient (as recipient of the prescription) → Recipient: Appropriate for the first sentence, where "to patient" marks the beneficiary of the prescribing event.
- Headache → Symptom: Appropriate and clinically meaningful — "Symptom" is a domain-specific role extension beyond generic Theme, valuable precisely because generic SRL roles do not capture clinical significance on their own.
- Medicine → Instrument: Questionable/inconsistent depending on which sentence it is drawn from. In "Doctor prescribed medicine to patient", Medicine is correctly a Theme (the thing transferred), not an Instrument (Instrument would be more appropriate for something the Doctor used to perform the prescribing, e.g., a prescription pad or e-prescribing system). In "Medicine reduced blood pressure", Medicine is grammatically the Subject and semantically the causal Agent/Instrument of the reduction — here "Instrument" is defensible only if one adopts the convention that a non-volitional causer is labelled Instrument rather than Agent, which is a common but not universal SRL convention.

Overall verdict: three of the four role labels (Agent, Recipient, Symptom) are appropriate and consistent with standard and domain-adapted SRL practice. The Medicine → Instrument label is the weakest of the four: it is defensible in the causal-verb sentence ("Medicine reduced blood pressure") but is a mislabel if it was derived from the prescribing sentence, where Medicine should be Theme, not Instrument. This ambiguity should be flagged and resolved per-sentence rather than assigned as a single global Entity→Role table, since the same entity (Medicine) legitimately plays different roles in different sentences.

4.3 Task 3 — Potential Errors from Incorrect Parsing

22. Prepositional-phrase attachment ambiguity: "prescribed medicine to patient" could, under a faulty parse, attach "to patient" to the wrong head, causing the Recipient role to be dropped or misassigned, which would break any downstream check that verifies every prescription has an identified recipient.
23. Agent/Experiencer conflation: "Patient reported severe headache" places Patient in the Subject slot, but Patient is an Experiencer, not a volitional Agent in the classic sense; a syntax-only pipeline that blindly maps Subject→Agent risks miscategorising symptom-reporting events the same way it categorises treatment-administering events, obscuring the clinically important distinction between "who did something" and "who experienced something."
24. Passive-voice misparsing: clinical notes frequently use passive constructions ("Medicine was administered to the patient by the nurse"); a parser tuned only to active SVO will mis-map the surface Subject ("Medicine") to Agent instead of correctly identifying "Nurse" (in the by-phrase) as the true Agent, silently corrupting the Agent role.

25. Negation and modality scope errors: missing or mis-scoped negation ("Medicine did not reduce blood pressure") can be parsed with the correct SVO skeleton but the wrong polarity, causing a semantic role labeller to assert a causal Medicine→BloodPressure relationship that the sentence actually denies — a serious risk in a clinical adverse-event or efficacy-tracking context.
26. Coordination and ellipsis errors: sentences such as "Nurse monitored and medicated the patient" require correctly distributing the shared Subject/Object across two coordinated verbs; a shallow parser can merge or drop one predicate's arguments, silently losing an entire clinical event from the structured record.
27. Ambiguous modifier scope affecting severity: in "Patient reported severe headache", if "severe" is mis-attached elsewhere (e.g., as a general patient-status descriptor rather than modifying the specific headache symptom), the Symptom role's severity attribute — which can drive triage priority — is corrupted.

4.4 Task 4 — Recommendations to Improve Syntax-Driven Semantic Analysis

28. Use dependency parsing over shallow phrase-structure parsing: dependency trees explicitly encode head–modifier and subject/object relations (including from passive by-phrases), giving semantic role labelling a more robust syntactic foundation than a flat SVO template.
29. Fine-tune on domain-specific clinical treebanks and SRL corpora: general-purpose parsers trained on newswire text under-perform on clinical syntax (elliptical notes, abbreviations, passive voice); using biomedical-domain resources (e.g., clinical treebank annotations, i2b2/n2c2-style corpora) substantially improves both parsing and role-labelling accuracy.
30. Adopt a hybrid rule-based + neural SRL pipeline: combine deterministic clinical rules (e.g., known drug-name gazetteers, dosage-pattern regexes) with neural semantic role labelling models so that high-precision structured entities (drug names, dosages) are never left to statistical inference alone.
31. Explicit negation and modality detection modules: integrate a dedicated negation/uncertainty detector (e.g., NegEx-style algorithms or a fine-tuned negation-scope classifier) as a pre- or post-processing step, so that role assignment is never decoupled from polarity.
32. Sentence-level, not entity-level, role tables: assign semantic roles per sentence/event rather than aggregating a single global Entity→Role mapping across the document, directly addressing the Medicine→Instrument inconsistency identified in Task 2, since the same entity's role legitimately changes across different clinical statements.
33. Human-in-the-loop clinical validation and active learning: route low-confidence parses and role assignments to clinical reviewers (nurses/coders), feeding corrections back into model retraining, given the high real-world cost of an undetected polarity, attachment, or role error in a patient-safety context.

5. Consolidated Conclusion

Across all four case studies, one theme recurs: the correctness of an NLP-driven decision system is bounded not by how well it recognises surface vocabulary, but by how faithfully it preserves meaning-critical distinctions — predicate polarity in the telecom chatbot, machine/product dependency structure in smart manufacturing, sense selection in e-commerce search, and syntactic role assignment in healthcare reports. In every domain, the highest-impact errors were the subtle ones that surface-level word recognition alone could not catch: a flipped action (ACTIVATE vs. DEACTIVATE), a missing redundancy path (Gear via M1 vs. M3), a wrong brand-vs-object sense (Apple), and an inconsistent global role label (Medicine as Instrument vs. Theme).

Two cross-cutting recommendations emerge for any organisation deploying semantic NLP systems at scale: first, pair statistical/contextual models (embeddings, click-log supervision, SRL neural networks) with formal or rule-based sanity checks (canonical intent ontologies, predicate logic consistency rules, negation detectors) so that each layer catches the class of error the other is blind to; and second, build continuous human-in-the-loop feedback into every pipeline, since all four case studies show that the most damaging errors are rare, high-impact edge cases that only sustained monitoring and periodic re-training reliably catch before they reach the end user.